LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL IV DAN V

IMPLEMENTASI OOP ENCAPSULATION

Disusun Oleh:

Ni Putu Haruka Ikeda 105222039

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Sistem reservasi hotel merupakan bagian penting dalam operasional perhotelan, namun sering terjadi masalah seperti jumlah tamu yang melebihi kapasitas kamar dan penyalahgunaan voucher.Hal ini biasanya disebabkan oleh kurangnya validasi data dan pengaman akses terhadap atribut dalam sistem.

Untuk mengatasi masalah tersebut, digunakan konsep Object Oriented Programming (OOP) khususnya prinsip Encapsulation, yaitu menyembunyikan data agar tidak dapat diakses langsung dari luar kelas serta mengontrol akses melalui method tertentu.

Pada praktikum ini, dibuat sebuah kelas KamarHotel yang menerapkan Encapsulation dengan atribut privat, constructor, serta method getter dan setter yang dilengkapi validasi. Selain itu, digunakan juga method overloading untuk proses pemesanan kamar dan perhitungan tagihan, sehingga sistem menjadi lebih aman dan terkontrol.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	*nomorKamar*	String	Menyimpan nomor identitas dari setiap kamar hotel
2	*tipeKamar*	String	Menyimpan jenis kamar (misalnya Reguler, Premium, atau Suite)
3	*kapasitasMaksimal*	Int	Menentukan jumlah maksimal tamu yang dapat ditampung dalam satu kamar
4	*hargaPerMalam*	double	Menyimpan harga sewa kamar per malam
5	*isTersedia*	Boolean	Menunjukkan status ketersediaan kamar (true = tersedia atau false = terisi)

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	*KamarHotel()*	*Constructor*	Inisialisasi data kamar (nomor, tipe, kapasitas, harga) dan set status ke"Tersedia".

2	getNomorKamar()	Functional (Getter)	Mengambil nilai nomor kamar
3	getTipeKamar()	Functional (Getter)	Mengambil nilai tipe kamar
4	getKapasitasMaksimal()	Functional (Getter)	Mengambil kapasitas maksimal kamar
5	getHargaPerMalam()	Functional (Getter)	Mengambil harga per malam
6	isTersedia()	Functional (Getter)	Mengambil status ketersediaan kamar
7	setTipeKamar()	Procedural (Setter)	Mengubah tipe kamar
8	setHargaPerMalam()	Procedural (Setter)	Mengubah harga per malam
9	pesanKamar()	Procedural	Memproses pemesanan kamar dengan mengecek ketersediaan dan kapasitas tamu
10	batalPesan()	Procedural	Membatalkan pemesanan dan mengubah status kamar menjadi tersedia kembali
11	hitungTotalBayar()	Functional	Menghitung total biaya berdasarkan jumlah malam menginap
12	tampilkanInfo()	Procedural	Menampilkan informasi lengkap mengenai kamar hotel

IV. Dokumentasi dan Pembahasan Code

4.1 Deklarasi Class dan Atribut

```
public class KamarHotel {

    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Class KamarHotel dirancang menggunakan prinsip enkapsulasi, di mana semua atributnya bersifat private sehingga tidak dapat diakses secara langsung dari luar. Objek ini menyimpan informasi mendetail terkait unit kamar, mulai dari nomor, tipe, dan kapasitas maksimal tamu, hingga variabel harga per malam. Selain itu, terdapat variabel boolean isTersedia yang berfungsi sebagai indikator status untuk memantau apakah kamar tersebut siap digunakan atau sedang terisi.

4.2 Constructor

```
public KamarHotel (String nomorKamar, String tipeKamar, int
kapasitasMaksimal, double hargaperMalam) {
        this.nomorKamar = nomorKamar;
        this.tipeKamar  = tipeKamar;
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = hargaperMalam;
        this.isTersedia = true;
    }
```

Dalam proses pembuatannya, class ini menggunakan Constructor yang akan dipanggil secara otomatis setiap kali objek baru diinisialisasi. Melalui kata kunci this, data yang dimasukkan akan langsung disimpan ke dalam atribut objek, seperti memetakan nomor kamar secara spesifik. Selain itu, status isTersedia secara otomatis diatur ke nilai true sebagai nilai bawaan (*default*), memastikan bahwa setiap kamar yang baru didaftarkan dalam sistem langsung berstatus siap untuk dipesan.

4. 3 Getter

```
public String getNomorKamar() {return nomorKamar;}
public String getTipeKamar () {return tipeKamar; }
public int getKapasitasMaksimal () {return kapasitasMaksimal;}
public double getHargaPerMalam() {return hargaPerMalam;}
public boolean isTersedia () {return isTersedia;}
```

Untuk mengakses informasi tersebut, class ini menyediakan metode Getter yang berfungsi sebagai jembatan untuk mengambil nilai dari atribut yang ada. Karena penerapan enkapsulasi membuat variabel bersifat tertutup (private), akses langsung dari luar kelas tidak dimungkinkan demi keamanan data. Oleh karena itu, setiap pengambilan data seperti mengecek harga atau tipe kamar harus dilakukan melalui metode ini, sehingga integritas data tetap terjaga dan terkontrol dengan baik.

4. 4 Setter

```
public void setTipeKamar (String tipeKamar){
    this.tipeKamar = tipeKamar;
}

public void setHargaPerMalam (double hargaPerMalam){
    this.hargaPerMalam = hargaPerMalam;
      }
```

Selain untuk mengambil data, class ini juga dilengkapi dengan metode Setter yang memungkinkan pengubahan nilai atribut secara aman setelah objek dibuat. Metode ini sangat krusial untuk memperbarui informasi kamar yang bersifat dinamis, seperti saat melakukan perubahan kategori pada tipeKamar atau menyesuaikan hargaPerMalam mengikuti kebijakan hotel.

4. 5 Method pesanKamar()

```
public void pesanKamar (int jumlahTamu) {
  if (isTersedia) {
      if (jumlahTamu <= kapasitasMaksimal) {
          isTersedia = false;
          System.out.println("Kamar telah dipesan");
      }else {
          System.out.println("tidak bisa, kapasitas penuh");
      }
      }else{
          System.out.println("Kamar sudah ada");
      }
              }
```

Proses pemesanan dimulai dengan memvalidasi ketersediaan kamar melalui status isTersedia. Jika kamar tersedia, sistem akan memverifikasi apakah jumlah tamu yang diajukan memenuhi syarat dengan membandingkannya terhadap kapasitasMaksimal. Pesanan akan dinyatakan berhasil hanya jika kedua syarat tersebut terpenuhi; namun, jika jumlah tamu melebihi batas atau kamar sudah terisi, maka permintaan pemesanan akan secara otomatis ditolak.

4. 6 Method batalPesan()

```
public void batalPesan () {
        isTersedia =true;
        System.out.println("Kamar dibatalkan");
    }
```

Untuk menangani pembatalan, terdapat metode khusus yang berfungsi mengembalikan status kamar menjadi tersedia kembali. Saat proses pembatalan dilakukan, variabel isTersedia akan diatur ulang ke nilai true, sehingga sistem secara otomatis mencatat bahwa kamar tersebut siap untuk dipesan oleh tamu lain. Langkah ini memastikan manajemen inventaris hotel tetap akurat dan dinamis dalam merespons perubahan reservasi.

4. 7 Method hitungTotalBayar()

```
public double hitungTotalBayar (int malam){
      return malam * hargaPerMalam ;
        }
```

Sistem memiliki fungsi untuk menghitung total biaya menginap berdasarkan durasi kunjungan tamu. Perhitungan ini dilakukan dengan mengalikan jumlah malam dengan harga per malam yang tersimpan dalam atribut objek. Hasil kalkulasi tersebut memberikan nilai akhir yang harus dibayarkan secara transparan, memudahkan proses administrasi dan penagihan saat tamu melakukan transaksi.

4. 8 Method tampilkanInfo()

```
public void tampilkanInfo () {
        System.out.println("Nomor Kamar : " + nomorKamar);
        System.out.println("Tipe Kamar : " + tipeKamar);
        System.out.println("Kapasitas : " + kapasitasMaksimal);
        System.out.println("Harga : " + hargaPerMalam);
        System.out.println("Status : " + (isTersedia ? "Tersedia " : "Terisi"));
        System.out.println("--------------------------");
    }
        }
```

Untuk menyajikan data secara komprehensif, terdapat fungsi yang menampilkan seluruh informasi kamar dengan memanfaatkan operator ternary untuk efisiensi logika. Sistem melakukan pengecekan instan terhadap status boolean kamar: jika bernilai true, maka informasi yang muncul adalah "Tersedia", namun jika false, secara otomatis akan tampil sebagai "Terisi".

V. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep Object Oriented Programming (OOP) khususnya prinsip Encapsulation sangat penting dalam membangun sistem yang aman dan terstruktur. Dengan menggunakan atribut bertipe private serta method getter dan setter, data dalam class dapat dilindungi dari akses langsung yang tidak diinginkan.

Selain itu, penggunaan method seperti pesanKamar(), batalPesan(), dan hitungTotalBayar() membantu dalam mengelola proses reservasi kamar secara lebih terkontrol. Sistem juga mampu melakukan validasi sederhana, seperti memastikan kapasitas kamar tidak terlampaui dan status kamar diperbarui dengan benar.

Dengan demikian, program yang dibuat dapat meminimalisir kesalahan operasional dan memberikan dasar yang baik dalam pengembangan sistem reservasi hotel yang lebih kompleks.

VI. Daftar Pustaka

Maiga, J., & Emanuel, A. W. R. (2019). Gamification for Teaching and Learning Java Programming for Beginner Students-A Review. J. Comput., 14(9), 590-595.

Wiener, R., & Pinson, L. J. (2000). *Fundamentals of OOP and data structures in Java*. Cambridge University Press.

Poo, D., Kiong, D., & Ashok, S. (2007). *Object-oriented programming and Java*. Springer Science & Business Media.